

SDDs from AI Loops: Creating Software Development Documents

Brian King

DigitalCoreNZ — <https://DigitalCoreNZ.com>

v0.5.6 — 03 August 2026

Abstract

This paper presents the sixth revision of the Software Development Documents SKILL for Hermes Agent, an autonomous AI agent framework. The system is built around four hierarchical entities—the Hermes Agent User, Hermes Agent, the Project Manager, and the Software Developer. The Project Manager and the Software Developer collaborate through a structured Refinement Loop to produce seven Software Development Documents from an Application Specification (AppSpec) file. The key is the integration of three components—Role Definitions, Watch and Control Modes, and Recovery Mechanisms. These interdependent components work together to form a complete, layered architecture. Every Entity in the hierarchy, who does not have Control, Watches for failure events, time-outs, and triggers from the Entity immediately below. There are prescribed recovery paths, made of recovery actions, that are followed by the Entity above the failure event. Watch Mode and Control Mode are the procedural foundations on which the Recovery Mechanisms are built. These Mechanisms are embedded directly into every Entity's Role Definition.

Keywords: dual-coders, Business Modelling Document, Business Requirements Document, Product Requirements Document, Context Requirements Document, User Interface Document, User Experience Document, Software Specifications Document, Role Definitions, Background Agents, Software Development Document, Project Manager, Software Developer

1. Introduction

The increasing sophistication of large language models (LLMs) and autonomous agent frameworks has opened new possibilities for software development process automation. The **Software Development Documents SKILL**—hereafter *SDDs from AI Loops*—is a procedural capability for **Hermes Agent** [1] that orchestrates the creation of a complete software development document set from a single Application Specification (AppSpec) file.

This paper documents revision v0.5.6, which introduces a formalised recovery architecture based on two complementary operating modes—*Watch Mode* and *Control Mode*—and a set of

Recovery Mechanisms that translate four canonical failure events into concrete, entity-level procedures. The architecture ensures that every failure has a detection path and a prescribed resolution action held by the appropriate hierarchical authority.

1.1 Related Work

Our work builds on the *dual-coders* pattern [2], in which a **Project Manager** and **Software Developer** collaborate through structured delegation to produce software artefacts. This pattern is extended here by the addition of explicit recovery semantics, layered observability, and bounded-convergence strategies for iterative refinement loops. The approach is complementary to agentic software engineering frameworks such as AutoGen [3]

and CrewAI [4], which also employ hierarchical agent teams but do not formalise recovery mechanisms at the level of individual entity role definitions.

1.2 Document Scope

The complete workflow produces seven documents in strict order, each dependent on the previous one:

1. **Business Modelling Document** — business process visualisation, organisational structures, domain models.
2. **Business Requirements Document** — high-level goals and success criteria (“what” not “how”).
3. **Product Requirements Document** — feature lists, user stories, functional requirements.
4. **Context Requirements Document** — system environment, external interfaces, boundaries.
5. **User Interface Document** — visual design, layout, wireframes.
6. **User Experience Document** — interaction patterns, user journeys.
7. **Software Specifications Document** — database schemas, API definitions, class diagrams.

Supporting reference files—

Dev_Docs_v0.5.1.md (document definitions) and Dev_Team_v0.5.1.md (team role definitions)—are loaded at runtime by both the Project Manager and the Software Developer.

2. System Architecture

The system comprises four **Entities** arranged in a strict hierarchy. Each entity holds a *Role Definition* that specifies its expertise, personality, activities, authorities, and recovery responsibilities.

2.1 Hierarchy and Entity Roles

1. **Hermes Agent User** — activates the SKILL, provides the AppSpec file path, holds final authority to approve documents or

terminate the workflow. Operates in *Watch Mode* by default, switching to *Control Mode* only when alerted to unresolvable conditions.

2. **Hermes Agent** — spawns the **Project Manager**, passes role definitions and control, and monitors the workflow for error envelopes, timeouts, or direct escalation messages. Responsible for respawning the Project Manager with last-known-good context and applying bounded-convergence directives.
3. **Project Manager** — spawns the **Software Developer**, creates and refines Software Development Documents, holds termination authority over the *Refinement Loop*. Draws on Business Analysis, Systems Analysis, and Project Architecture experience.
4. **Software Developer** — receives documents and control, produces revised versions, triggers escalation when it encounters unresolvable conditions. Draws on Development Management, QA, Security Engineering, DevOps, and UI/UX Design experience.

2.2 Role Definition Example

The **Project Manager**'s role definition is provided verbatim to the spawned agent as a system prompt:

Project Manager Role Definition (abbreviated)—“You are an expert Project Manager with many years of experience leading software development teams. Over time, you have also assumed the roles of Business Analyst, Systems Analyst, and Project Architect. Your current task is to spawn, and work with, the Software Developer. You create, and help refine, the Software Development Documents that turn an AppSpec file into a guided plan for creating deployable applications and utilities.”

The complementary **Software Developer** role definition is symmetrically structured, granting technical implementation authority while

withholding termination and progression authorities—a deliberate asymmetry that prevents the refinement process from stalling.

3. Refinement Loop

The explicit role definitions enable a fully *self-orchestrated refinement* process. The Project Manager and Software Developer drive the entire sequence without requiring Hermes Agent to track intermediate versions. The loop proceeds as follows for each of the seven documents:

1. Project Manager evaluates the AppSpec file.
2. Project Manager creates the numbered subdirectory and initial document (e.g. `./01_BMD/BMD_v0.5.1.md`).
3. Project Manager passes the initial document and *Control* to the Software Developer, then switches to *Watch Mode*.
4. Software Developer evaluates the document against the AppSpec file.
5. Software Developer creates a revised version with an even version number (e.g. `v0.5.2`) incorporating technical implementations.
6. Software Developer passes the revised document and *Control* back to the Project Manager.
7. Project Manager evaluates the revision against both the original and the AppSpec file.
8. Project Manager decides to create another revision (odd version, e.g. `v0.5.3`) or declare the document ready.
9. If ready, the Project Manager sends the document and *Control* to Hermes Agent.
10. Hermes Agent either sends requirements for another loop iteration or sends approval.
11. On approval, the Project Manager advances to the next document.

3.1 Version-Number Convention

The version-number progression creates a complete audit trail:

- **Odd** version numbers (v0.5.1, v0.5.3, ...) are produced by the **Project Manager**.
- **Even** version numbers (v0.5.2, v0.5.4, ...) are produced by the **Software Developer**.

This convention ensures that every revision is traceable to its originating entity.

3.2 Staged Approval

The Project Manager does not begin work on the next document until Hermes Agent approves the current one. This staged approval prevents wasted work on unstable foundations—each document depends on the previous one, so premature progression would compound errors through the document chain.

4. Watch Mode and Control Mode

The two operating modes form the procedural infrastructure for observation and intervention.

4.1 Control Mode

When an entity holds *Control*, it has exclusive authority to perform actions within its scope of influence. Passing *Control* to another entity relinquishes all action capability. *Control* is passed sequentially down the hierarchy: **Hermes Agent User** → **Hermes Agent** → **Project Manager** → **Software Developer**, and back up as documents are completed.

4.2 Watch Mode

Entities *without* *Control* automatically observe the entity directly beneath them in the hierarchy. This observability chain is the structural reason the entities are arranged hierarchically—it creates a layered detection surface:

- **Software Developer** → observed by → **Project Manager**
- **Project Manager** → observed by → **Hermes Agent**

- **Hermes Agent** → observed by → **Hermes Agent User**

4.3 Watch Mode as Recovery Foundation

Watch Mode is the detection layer that enables every recovery action. When a trigger condition is detected, the watching entity reclaims Control and executes the prescribed recovery action. The trigger chain flows upward through the hierarchy:

- **Software Developer → Project Manager:**
The Software Developer signals a structured alert when it encounters an unresolvable condition (missing file, invalid schema, transient failure, version mismatch). The Project Manager detects this alert in Watch Mode, switches to Control Mode, and either respawns (transient, max 3 retries) or escalates (persistent).
- **Project Manager → Hermes Agent:** The Project Manager signals an alert when the Refinement Loop fails to converge, the Software Developer fails beyond the retry limit, or the Project Manager encounters an error it cannot resolve. Hermes Agent detects this in Watch Mode and respawns the Project Manager with context, applies a bounded-convergence directive, or escalates to the User.
- **Hermes Agent → Hermes Agent User:**
Hermes Agent signals an alert when a recovery action has failed, the document cannot be completed due to unresolvable AppSpec errors, or human judgment is required. The Hermes Agent User detects this in Watch Mode and provides direction, approves, or terminates.

5. Recovery Mechanisms

The Recovery Mechanisms translate four canonical failure events into concrete procedures embedded in the role definitions. Each failure event is detected through Watch Mode and resolved through Control Mode intervention.

Table 1: Recovery Mechanism Matrix

Failure Event	Watch Detection	Control Recovery Action
PM fails mid-Refinement Loop	HA observes error envelope	Respawn PM with last-good doc path as context; log failed prompt_id
SD fails mid-revision	PM observes error envelope from child	PM decides respawn (transient, max 3) vs escalate (persistent)
Indefinite Refinement Loop	PM observes non-convergence; HA observes timeout	PM applies bounded strategy (10 rounds / 5% delta); HA respawns with directive if PM fails
Version-number collision	Receiving entity observes missing-file or mismatch	Validate file chain via version-history metadata; request corrected submission

5.1 Transient vs Persistent Failure Handling

The recovery architecture distinguishes two failure classes:

Transient failures

Network blips, model timeouts, transient execution errors. These are handled by respawning the affected entity for the same version, up to three retries, without escalating.

Persistent failures

Missing files, invalid schemas, permission-denied errors. These escalate to the next entity in the hierarchy with a description and the last-known-good document version. The escalation terminates the refinement for that document until human intervention resolves the root cause.

5.2 Bounded Convergence

An indefinite refinement loop—where successive revisions do not converge—is bounded by two criteria:

1. **Round limit:** stop after ten (10) refinement rounds.
2. **Delta threshold:** stop when successive versions differ by less than five percent (5%) measured by structural diff.

When either criterion is met, the Project Manager passes the best result to Hermes Agent. If the Project Manager itself fails to enforce the bound, Hermes Agent respawns it with a bounded-convergence directive.

6. Directory Structure Convention

Each document lives in its own numbered subdirectory created under the AppSpec path:

```
[AppSpec_File]/
├── 01_BMD/
│   └── BMD_v0.5.1.md
├── 02_BRD/
│   └── BRD_v0.5.1.md
├── 03_PRD/
│   └── PRD_v0.5.1.md
├── 04_CRD/
│   └── CRD_v0.5.1.md
├── 05_UID/
│   └── UID_v0.5.1.md
├── 06_UXD/
│   └── UXD_v0.5.1.md
└── 07_SSD/
    └── SSD_v0.5.1.md
```

The Project Manager creates each subdirectory and the initial document file. Both the Project Manager and the Software Developer increment version numbers during each Refinement Loop. The project path is the directory containing the AppSpec file unless the user specifies otherwise.

7. Analysis

The integration of Watch Mode and Control Mode as the foundation for Recovery Mechanisms—embedded directly into the Role Definitions of every entity—is the critical advancement in revision v0.5.6. This analysis

evaluates the three interdependent components and demonstrates that each is necessary for the others to function.

7.1 Role Definitions (v0.5.4 Foundation)

The Role Definitions establish expertise, personality, activities, and authorities for each entity—boundaries that define who may act, who may decide, and who must escalate. Without these definitions, the trigger conditions that activate Watch and Control have no context: an entity cannot detect an anomaly if its role definition does not specify normal operation.

7.2 Watch and Control (v0.5.5 Foundation)

Watch and Control provide the procedural mechanism for observability and authority transfer. Watch assigns automatic observation of the entity directly beneath each watcher. Control grants exclusive action authority to the holder. When a trigger is detected during Watch, the watcher reclaims Control—this transfer is the structural precondition for every recovery action.

7.3 Recovery Mechanisms (v0.5.6 Contribution)

The Recovery Mechanisms are concrete actions embedded in each entity's role definition:

- **Software Developer** carries structured alerts.
- **Project Manager** carries respawn-vs-escalate logic with a retry counter.
- **Hermes Agent** carries detection-and-respawn protocol.
- **Hermes Agent User** carries final human authority.

Every recovery action is triggered by Watch Mode detection and executed through Control Mode intervention.

7.4 Interdependence

Role Definitions without Watch and Control provide boundaries but no intervention mechanism. Watch and Control without Recovery Mechanisms provide observation but no prescribed actions. Recovery Mechanisms

without Role Definitions provide actions but no trigger context. Together they form a complete recovery architecture where:

- Every entity watches the entity beneath it.
- Every failure event has a prescribed recovery path.
- Every recovery action is executed by the entity with appropriate authority.

8. Conclusion

Revision v0.5.6 of **SDDs from AI Loops** achieves three interdependent objectives that establish a complete recovery architecture for AI-orchestrated software development document generation:

1. **Role Definitions** (v0.5.4) — establish expertise, personality, activities, and authorities, creating the boundaries against which anomalies are detected.
2. **Watch and Control** (v0.5.5) — provide the procedural infrastructure for observation and intervention; the mode transfer triggered by anomaly detection is the operational backbone of every recovery action.
3. **Recovery Mechanisms** (v0.5.6) — translate the four failure events into prescribed actions embedded in each entity's role definition.

The three components are interdependent: Role Definitions without Modes provide no intervention; Modes without mechanisms provide no actions; mechanisms without definitions provide no trigger context. Together they form a layered architecture in which Watch Mode is the foundational detection layer—every recovery action begins with a Watch Mode observation and proceeds through a Control Mode intervention. The hierarchy exists specifically to enable this chain of observation, trigger, and response.

Acknowledgements

The author thanks the Hermes Agent development team and the Nous Research community for providing the agentic infrastructure that made this SKILL possible.

References

- [1] Nous Research. *Hermes Agent — An Intelligent AI Assistant*. 2026.
- [2] B. King. Internal document called “Dual Coders: A Pattern for Autonomous Software Development Using Agent Teams.” DigitalCoreNZ Technical Report, 2026.
- [3] AutoGen. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. Microsoft Research, 2023.
- [4] CrewAI. *CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents*. 2024.